

EECS470 Final Report

A P6-style Out-of-Order CPU Design

Lingxiao Yang
EECS Department
University of Michigan
Ann Arbor, USA
yanglx@umich.edu

Runshuang Guo
EECS Department
University of Michigan
Ann Arbor, USA
guorsroy@umich.edu

Rui Jiang
EECS Department
University of Michigan
Ann Arbor, USA
jiangrex@umich.edu

Juntao Wu
EECS Department
University of Michigan
Ann Arbor, USA
jtwuuuuu@umich.edu

Abstract—Our design is a 3-way superscalar out-of-order CPU with register renaming, a reservation station, LSQ, multiple functional units, reorder buffer, and separate I/D caches. The pipeline supports multiple in-flight memory operations, aggressive speculation LSQ, and non-blocking memory execution.

Index Terms—Out-of-Order Processor, 3-way Superscalar, Branch Prediction, Prefetching, Non-blocking Cache, Speculation load/store Queue

I. INTRODUCTION

This project implements a 3-way superscalar out-of-order RISC-V processor built on a streamlined P6-style pipeline. The design integrates reorder buffer, a reservation station, a map table, a load/store queue (LSQ), and separate instruction and non-blocking data caches.

Major enhancements include improved branch prediction, multi-block prefetching, aggressive LSQ forwarding validated via the CDB, and a multi-threaded non-blocking data cache that sustains multiple outstanding misses. A GUI debugger and extensive module-level testing support correctness and performance tuning.

The team collaborated across major architectural blocks with clearly partitioned responsibilities:

- **Juntao Wu** designed the ROB, instruction fetch unit, and LSQ, implemented the final version of the data cache, and participated in CPI and timing optimization.
- **Lingxiao Yang** developed the map table, instruction cache, and branch predictor, and conducted overall processor configuration tuning.
- **Runshuang Guo** designed the reservation station, implemented the initial data cache, contributed to final-cache debugging, and participated in CPI and timing optimization.
- **Rui Jiang** designed the functional units and CDB subsystem and supported integration and backend verification.

The project also faced several notable challenges. Improving branch-prediction performance, particularly the BTB hit

rate, was essential to reducing misprediction rate and CPI. Debugging and refining the non-blocking data cache, along with victim-cache behavior and concurrency control which end up with failure, required careful coordination with the LSQ and memory pipeline. Meeting synthesis timing constraints demanded targeted critical-path reduction, especially along the LSQ → D\$ → CDB and RS datapaths. Despite these constraints, the final processor passes all correctness tests, achieves a relatively low clock cycle, and demonstrates competitive CPI across the benchmark suite.

II. PROJECT DESIGN

A. CPU Overview

1) *Structure Overview*: The basic structure and signal transition flow chart are shown in the figure below.

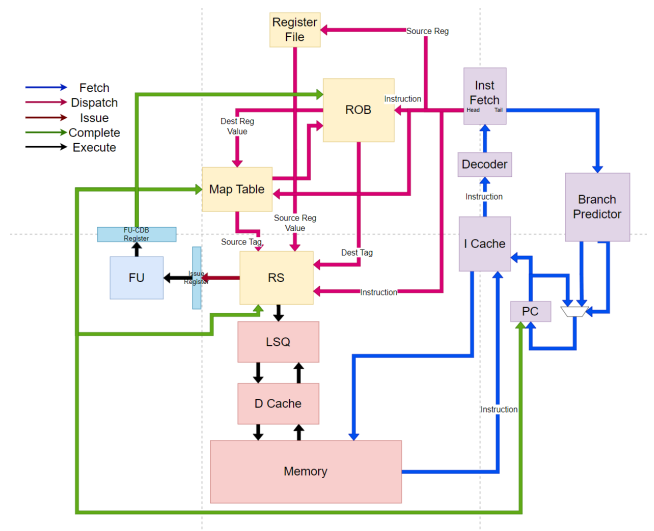


Fig. 1. CPU Architecture Overview

2) List of Features:

Basic Features

Implements a P6-style out-of-order pipeline with register renaming, RS-based scheduling, multi-FU execution, branch prediction, LSQ-based memory ordering, and instruction/data caches

Advanced Features

- 3-way superscalar issue
- LSQ forwarding and speculative load execution
- Sophisticated branch predictor (bimodal + GShare + chooser) with return address stack
- Multi-block instruction prefetching
- Fully non-blocking data cache
- Out-of-order memory issue via LSQ
- Comprehensive unit testbenches
- GUI debugger

B. Reservation Stations

Our 3-way out-of-order pipeline uses a reservation station (RS) to buffer renamed instructions until their operands become ready and an appropriate functional unit (FU) or LSQ becomes available. Each RS entry stores the operation class, destination ROB tag, two source operands with readiness and tags, and decoded control information including the instruction word, ALU function, operand selects, and PC/NPC. A compact RS state packet is exported for GUI debugger implementation and performance monitoring.

Logically, each RS entry contains:

- a busy bit indicating whether the entry is allocated,
- an `opclass` encoding the operation class (ALU, multiply, load, store, branch), which determines which FU type or LSQ path may consume the entry,
- a `rob_tag` identifying the destination ROB entry for commit and CDB tagging,
- for each source operand: a readiness bit, producer ROB tag, and data value (`s1_ready`, `s1_tag`, `s1_val`; `s2_ready`, `s2_tag`, `s2_val`),
- decoded control fields including the instruction word, ALU function, operand-select signals, PC, NPC, and the unconditional-branch flag.

Instructions enter the RS from up to three dispatch lanes per cycle. Dispatch determines operand readiness using the Map Table and ROB: if a source corresponds to an architectural value or a completed ROB entry, it is marked ready and its value is supplied directly; otherwise, the producer's ROB tag is recorded and the operand is marked not ready. A priority selector allocates free RS entries each cycle, using a free-entry mask that incorporates both currently unused entries and entries that will be freed by issue in the same cycle. When a lane receives a grant, its RS entry is filled with the dispatch metadata and the decoded control information. At dispatch time the RS also snoops the CDB: if a dispatched source tag matches a value currently being broadcast, the operand is immediately marked ready and the forwarded value is captured, avoiding an extra wakeup cycle. Per-lane dispatch-ready signals apply backpressure when no entries are available.

Operand readiness is updated dynamically via CDB wakeup. Each cycle the RS compares all valid CDB tags against the pending source tags of busy entries. When a match is found for a not-ready source, the operand is marked ready, the broadcast value is written into the entry, and the tag is cleared. Once both operands of an entry are ready, the instruction becomes eligible for selection. The RS interface also includes early-tag wiring intended for an Early Tag Broadcast feature; although ETB is disabled in the current configuration, the RS structure and fields are designed so that early wakeup can be enabled without structural changes.

Selection is performed per operation class (ALU, multiply, load, store, branch). For each class, a dedicated selector chooses up to as many ready entries as there are FUs of that type. The selected entries generate issue packets containing the source values, ROB tag, and decoded control signals (instruction, ALU function, operand selects, PC/NPC, branch flags). To reduce the critical path from wakeup-select to FU execution, each FU lane includes a small issue register that implements a standard valid/ready handshake. The RS issues into a lane only when the selector chooses that entry and the corresponding issue register is ready to accept new data.

To enforce the global three-wide issue limit, we add a second arbitration stage across all FU lanes. Lanes whose issue registers hold valid packets and whose FUs are ready form the candidate set; a global selector chooses up to three lanes to fire in the current cycle. Firing both advances the issue registers and frees the corresponding RS entries. Completed FU results are broadcast on the CDB and fed back into the RS, closing the wakeup-issue loop.

On reset or squash (after a mispredicted branch), all RS entries are cleared in a single cycle by resetting their busy bits. During normal operation, the RS maintains a running count of free entries by counting deasserted busy bits. This count is reported to the ROB for front-end backpressure and is also used in our performance evaluation to study RS occupancy and bottlenecks.

C. Reorder Buffer

ROB is implemented in the early stage of the project and remain free of bug throughout due to design decisions. The design, similar to LSQ and DCache, separate the implementation of entries with the dispatch logic from the main ROB central unit. Each entry has its own state machine. Each will change according to the input from CDB and control signals sent to ROB from CPU. For example, instead of deciding how a branch is mispredicted, the comparison happens inside the state machine dedicated to each entry. By separating the logic, the debug difficulty reduced dramatically.

Due to the simplistic design choice, optimizations for critical path is performed with ease. The dispatching logic became one of the critical paths as discussed in section V. By modifying a modular code segment, the clock period reduced to a final 8.3ns. The smooth transition all owe to the ROB's modularity nature.

D. Map Table

The map table keeps the architectural register namespace synchronized with speculative physical producers. Each entry tracks whether the register is currently renamed, which ROB tag is slated to produce it, and whether that producer has broadcast its value on the common data bus. Entry 0 ($\times 0$) is hardwired to “not renamed” so the zero register always bypasses renaming.

Every cycle we start from the registered state, apply CDB broadcasts, retirements, and then allocations. CDB processing walks the entire table and marks any entry whose ROB tag matches a valid broadcast as ready. Retirement packets clear the entry only if the retiring ROB index still owns that architectural register, preventing younger overwrites from being clobbered. Allocation packets then install new tags; lane ordering is from low to high index, so the latest lane in the same cycle wins when multiple instructions target the same architectural register.

Lookup logic serves the decode/issue pipeline. For each instruction lane we search the allocations issued earlier in the same cycle so that younger instructions can read tags created by older siblings. If no same-cycle allocation applies, we fall back to the registered map table contents. A ready bit accompanies every lookup: it is asserted when the source either maps to $\times 0$, hits a retiring instruction in-flight, or finds its tag on the CDB during the current cycle. Otherwise the consumer receives the ROB tag along with a `not_ready` flag, and downstream RS entries will wait for the tag to broadcast.

E. Common Data Bus and Functional Units

The functional units and common data bus form the core of the out-of-order backend. The `fu` block instantiates ALU, multiplier, and branch lanes, each exposing a uniform interface to the reservation stations and CDB. ALU lanes are combinational but wrapped in small one-entry buffers that hold the result and ROB tag until a CDB grant arrives. Operand-select multiplexers supply register values, PC/NPC, or immediates, allowing each ALU lane to handle arithmetic, address generation, or branch-target computation. An ALU lane issues only when its buffer is empty, and the same valid bit serves as both its “busy” flag and its CDB request.

Multiplier units are implemented as pipelined datapaths composed of `mult_stage` blocks. They prioritize throughput: once a multiply begins, its operands, ROB tag, and partial products flow through the stages until `done` asserts. The pipeline advances only when `stall` is low, so the CDB can safely back-pressure the multiplier without losing tag alignment. As long as the CDB is not stalling, the RS may issue a new multiply every cycle.

Branch execution uses a dedicated lane combining a condition checker and target adder. A `conditional_branch` unit resolves taken/not-taken, while an ALU instance computes the branch target. Like the ALUs, the branch unit buffers its result (ROB tag, fall-through PC, target, taken bit) until it receives a CDB grant, enabling unified handling of misprediction updates downstream.

The CDB arbitrates among all FU requesters and can broadcast up to `CDB_SZ` results per cycle. Request vectors are rotated by a round-robin pointer before entering `psel_gen`, ensuring fairness across busy units. After un-rotation, the CDB selects the corresponding FU results and drives them to the map table, ROB, RS, and PRF. Any unused broadcast slots are explicitly marked invalid.

Backpressure is conveyed through a simple stall/grant protocol. The CDB asserts `fu_stall` for any FU that requested but was not granted this cycle. ALU and branch units stop accepting new work until their buffered result is broadcast, while the multiplier pipeline freezes automatically on stall. This unified interface keeps FU datapaths simple while the CDB handles arbitration, fairness, and multi-way fan-out.

F. Packets Design

We rely on strongly typed “packets” to shuttle structured information between stages instead of wiring dozens of individual signals. Each packet is a packed struct defined in `packets.svh`, so widening or trimming signals becomes a single edit that propagates everywhere through the type system. This organization also keeps module interfaces readable.

One representative example is the `IFB_PACKET`. It bundles architectural source/destination registers, decoded control signals (ALU function, operand mux selects, memory bits), prediction metadata (taken/gshare/bimodal decisions plus *PHT* index), call/return hints for the Return Address Stack, and both PC/NPC addresses alongside the raw instruction. Because the entire bundle travels together, downstream stages can redo decoding or rebuild targets without chasing separate wires.

Some other typically used packets include:

- `FETCH2ROB_PACKET` for fetch-to-ROB allocation metadata
- `ROB2MAPTABLE_PACKET` for allocation and retirement updates
- `ROB2FETCH_PACKET` for branch-resolution feedback
- `MT_RS_PACKET` for map-table lookup results
- `RS_FU_PACKET` for operands issued to functional units

Together they keep interfaces uniform, reduce copy-paste signal lists, and provide a single source of truth for every data bundle flowing through the machine.

III. ADVANCED FEATURES

A. Instruction Cache and Prefetching

Our instruction cache targets a three-way superscalar fetch window while keeping the structure compact enough to hit timing. The cache is direct-mapped with 32 total lines of 8 bytes each. We split the array into even/odd banks so two consecutive 64-bit lines (PC and PC+8) can be delivered every cycle; the fetch unit consumes the first line immediately and opportunistically issues from the second when it needs a third instruction. Per-bank tag and predecode metadata (return bits, conditional-branch bits) mirror the data arrays so the front end never waits on instruction classification.

To reduce conflict pressure we append a four-entry victim cache. Each entry stores a full block, the original direct-mapped index, and the tag, and a tiny two-bit LRU counter approximates MRU/LRU behavior. Any miss that hits the victim cache forwards data in place, and eviction of a modified line routes through the victim cache before spilling to memory. This arrangement lets the direct-mapped banks stay small and fast while still capturing most two-way working sets.

The miss controller can sustain four outstanding transactions: the demand ports targeting PC and PC+8 plus two sequential prefetch streams pointed at PC+16 and PC+24. Demand requests always win arbitration. A prefetch launches only after a miss and only if the next sequential block is absent from both the main cache and the victim cache and no other in-flight request is already fetching that block. Prefetching continues down the sequential path while each prior prefetch completes successfully; if the base PC changes or if the target block is discovered in the cache or victim cache, the controller immediately cancels the speculative stream. Every in-flight request carries a unique transaction tag, allowing fills to arrive out of order while still writing into the correct bank and optionally capturing the evicted line for the victim cache.

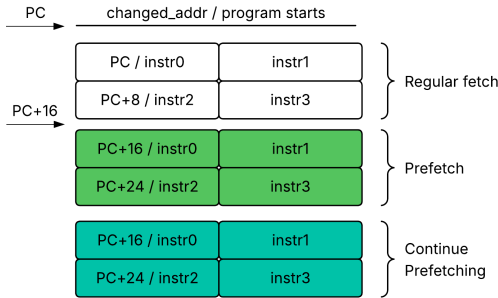


Fig. 2. Prefetching Logic Illustration

B. Instruction flow and Branch Prediction

The fetch stage drives the program counter into the instruction cache, gathers up to three instructions per cycle, and pushes them into the Instruction Fetch Buffer while consulting all prediction structures in parallel. The Instruction Cache supplies two aligned 64-bit lines, letting the stage assemble three consecutive 32-bit instructions (PC, PC+4, PC+8). The stage tracks how many IFB slots remain so it never over-issues fetches, and it stalls Behind cache misses until both lines arrive and the downstream buffer has space. When a redirect arrives from the commit side, the fetch PC is immediately overwritten with the branch target so all speculative instructions in flight are replaced.

1) *Return Address Stack*: We implement a 16-entry Return Address Stack that mirrors the three-wide fetch window. Speculative updates push call return addresses and pop returns as soon as the fetch stage decodes a lane tagged as call/return by the instruction decoder and predecode metadata. On a

pipeline recovery we copy the committed stack back into the speculative stack so any incorrect pushes or pops are squashed, and the top entry feeds the predicted target for return instructions when no other predictor information is available.

2) *Branch Predictor*: The branch predictor is a hybrid gshare plus bimodal design. Each lane forms a predicted PC by XORing the fetch PC with the 6-bit global history register, so the Pattern History Table holds 64 two-bit counters. The same PC bits also index a separate 64-entry bimodal table. A chooser table associated with each PC decides which predictor to trust for that lane, and decode feedback updates all three tables every time a conditional branch retires. The result is a per-lane taken/not-taken decision and a ready Pattern History Table index so the predictor can be updated when the ROB resolves the branch.

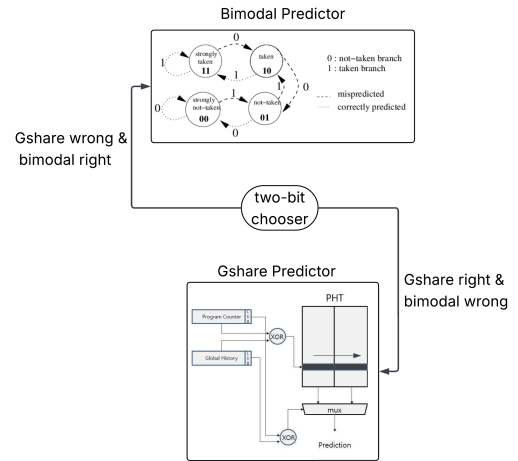


Fig. 3. Hybrid Branch Predictor

Accuracy comparison. Figure 4 compares average branch prediction hit rate under three configurations using a 6-bit global history. The bimodal-only baseline achieves 84.3%, while gshare-only improves to 85.4% by exploiting global correlation. The hybrid predictor reaches 87.2%, outperforming bimodal by 2.9 percentage points and gshare by 1.8 points. This gain comes from allowing the chooser to select bimodal for strongly biased branches and gshare for history-correlated branches, reducing aliasing and avoiding cases where a single predictor class performs poorly on certain control-flow patterns.

3) *Branch Target Buffer*: The Branch Target Buffer is a 64-entry direct-mapped array that stores the last observed target for every taken branch. For each fetch lane the BTB adds the lane offset to the base PC, extracts index and tag bits, and returns a hit signal plus target if the entry is valid. Resolved branches write their targets back into the BTB entry associated with the instruction's PC, so future occurrences can redirect the fetch PC without waiting for decode or execution results.

4) *Instruction Fetch Buffer*: The Instruction Fetch Buffer (IFB) is a 32-entry FIFO that decouples the front end from the decoder. The fetch stage writes up to three packets per

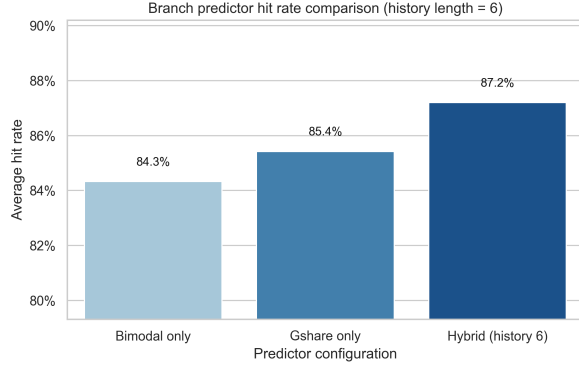


Fig. 4. Branch predictor hit rate comparison (history length = 6).

cycle, and the decode stage consumes up to three. Head/tail pointers track occupancy, and a “spots” signal fed back to the fetch stage ensures we never overflow the buffer. Each entry stores the instruction, its PC, and decoded meta-data (source/destination registers, classification bits, call/return hints), giving the downstream rename stage all information needed to continue issuing even when fetch stalls.

C. Data Cache

Our final Dcache is **Non-blocking, write-back, Direct-mapped**. Dcache has non-blocking and a Instead of doing an mshr, I am using the concept of *dcache threads* to allow full concurrency. Since Dcache is essentially a state machine interacting with the Dcache Memdp module and memory, dividing into multiple threads with each running their own statemachine is a natural way to achieve concurrency.

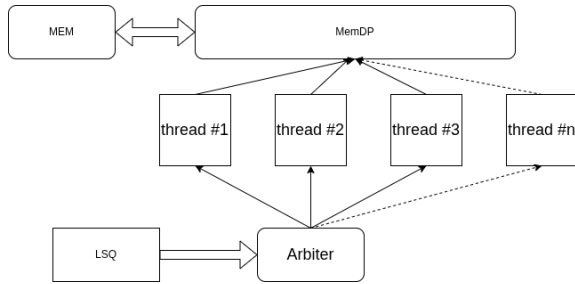


Fig. 5. DCache design methodology

Another benefit of this design decision is that we can easily adjust the level of concurrency by tweaking the parameters. The N_THREAD parameter controls how many dcache thread modules are instantiated. When maintaining the interface in CPU unchanged, the arbitrator will adjust with the parameter and arbitrate obstructed requests to free lines of threads.

When optimizing for synthesis, the state-machine-like design choice also brings much benefits. It helps us reckonize the critical path faster and mitigate with ease. More of this is discussed in section V

The most difficult part in achieving this is dealing with race condition when multiple entities are accessing the same cache line. It is very easy to modify without evicting, losing data.

We finally chose $N_THREAD=3$. It obtains the optimal CPI as is discussed in Section IV.

D. Load-Store Queue

Our LSQ ultimately adopts **optimally forwarding and internal speculative loads**. LSQ is implemented with the same methodology as DCache and ROB. Each entry in the Load Queue and Store Queue is a state machine as is shown in Figure 6. There is in total of five main states: *IDLE*, *WAIT_ADDR*, *WAIT_DATA*, *PENDING*, *COMMIT*. Whence got the data and addresses from Reservation Station, we would get out of the *WAIT_ADDR* state and enter *PENDING* state. In this state, we check 1) if there is any conflicts (chance of forwarding) 2) unresolved address from store queue, we would accordingly change our states to either 1) speculatively issue our loads 2) start CDB broadcasting if all the bytes are covered.

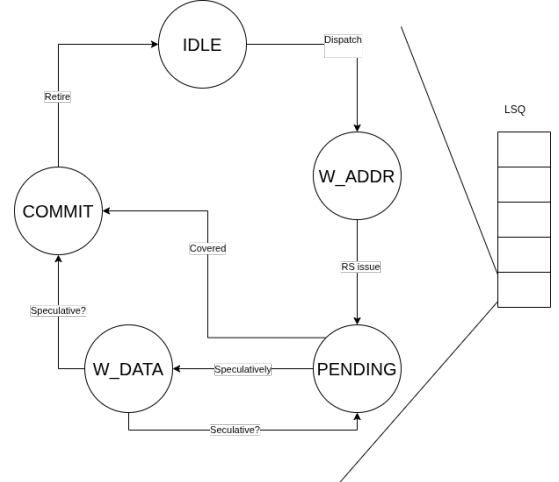


Fig. 6. LSQ design methodology

The addresses of store-queue entries needed for conflict analysis are forwarded to the load queue when they are issued from the Reservation Station. As a result, the “Age” detection logic is distributed across individual load entries, which greatly reduces the overall complexity of the LSQ. Although `LSQ.sv` exceeds 800 lines of code, it remains highly maintainable and readable.

Also, to maintain minimal design, we adopted ROB index as a way to detect the younger/older relation of instructions.

IV. RESULTS AND EVALUATION

Our final out-of-order core achieves a minimum clock period of **8.3 ns** based on post-synthesis timing analysis, and across all benchmarks it attains a weighted average CPI of **2.129**. These two numbers define the baseline performance of the design and contextualize the architectural sensitivity studies that follow.

Program	CPI
alexnet	3.186
backtrack	3.260
basic_malloc	4.752
bfs	3.230
dft	2.635
fc_forward	3.098
graph	3.791
insertionsort	1.168
matrix_mult_rec	6.359
mergesort	2.929
omegalul	4.838
outer_product	1.249
priority_queue	4.649
quicksort	2.133
saxpy	2.337
sort_search	1.446

TABLE I
CPI FOR ALL C PROGRAMS.

A. Impact of ROB Size

ROB size has a noticeable impact on performance. Figure 7 shows a clear reduction in weighted CPI as the ROB grows from 16 to 32 entries:

$$\text{WCPI}(16) = 2.280 \rightarrow \text{WCPI}(32) = 2.129.$$

This improvement reflects the larger instruction window, which exposes more ILP and allows execution to continue during stalls and cache misses. Beyond 32 entries, however, gains saturate:

$$\text{WCPI}(48) = 2.121, \quad \text{WCPI}(64) = 2.121,$$

indicating diminishing returns as other structures (RS, LSQ, predictor) become bottlenecks.

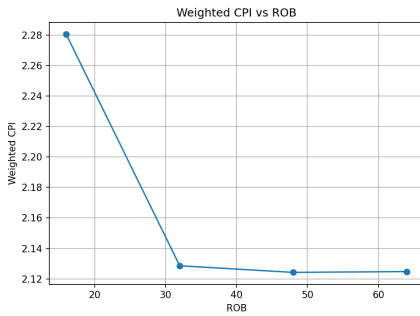


Fig. 7. Weighted CPI vs. ROB size.

B. Impact of Reservation Station Count

RS size has minor effect on global performance. Weighted CPI varies slightly from 2.123 (RS=8) to 2.136 (RS=20), with no consistent trend:

$$\text{WCPI}(8) = 2.123, \quad \text{WCPI}(20) = 2.136.$$

This suggests that RS sizes in this range do not significantly impact ILP extraction, and small variations reflect benchmarking noise rather than true architectural effects.

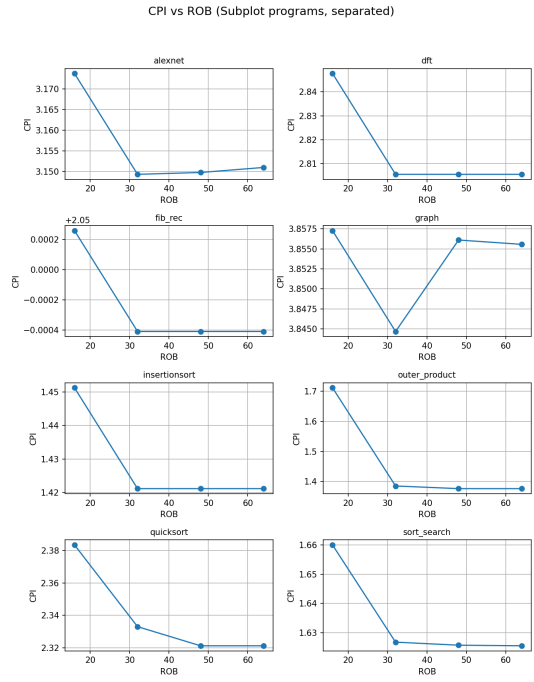


Fig. 8. Per-benchmark CPI sensitivity to ROB size.

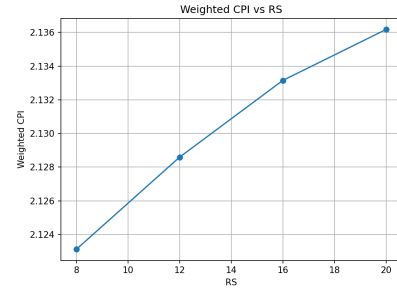


Fig. 9. Weighted CPI vs. RS entries.

C. Impact of Gshare History Length

We also evaluate how branch predictor history length affects prediction quality. Figure 10 shows per-benchmark Gshare hit rates for history lengths 4, 6, and 8.

Several patterns emerge:

- Regular control flow (e.g., `alexnet`, `fib_rec`) shows steady improvement with longer history.
- Hard-to-predict workloads (e.g., `graph`) show smaller gains.
- Highly predictable workloads (e.g., `outer_product`) exhibit nearly constant accuracy across all lengths.

Weighted hit rates follow a clear upward trend:

$$\text{WHR}(4) = 85.8\% \rightarrow \text{WHR}(8) = 88.3\%.$$

Longer history consistently improves global prediction accuracy, though with diminishing returns as aliasing effects begin to dominate.

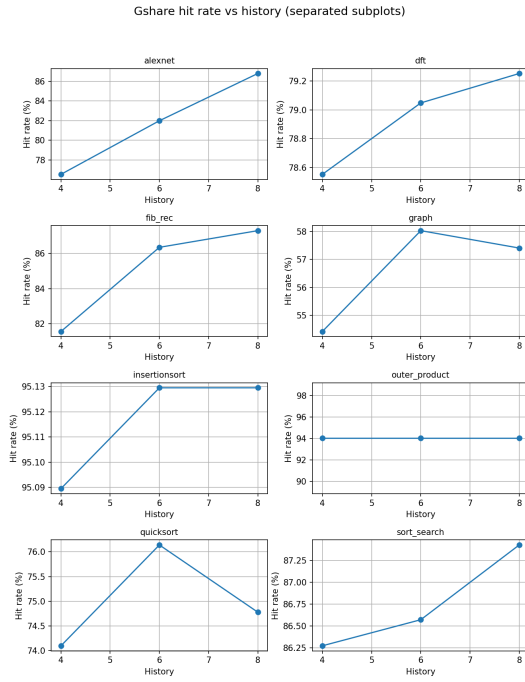


Fig. 10. Gshare hit rate vs. history length (per benchmark).

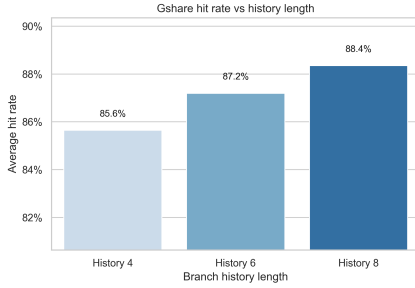


Fig. 11. Weighted Gshare hit rate vs. history length.

D. Impact of Prefetching

Figure 12 compares CPI for a subset of memory-intensive C benchmarks with and without the sequential i-cache prefetcher enabled. Prefetching consistently lowers CPI for every workload except `sort_search`, with the largest reductions appearing in `alexnet`, `basic_malloc`, and `priority_queue`.

The bar chart highlights that the prefetcher mainly helps workloads with streaming or stride-friendly access patterns, while irregular traversals such as `graph` still see a meaningful but smaller benefit. The only regression (`sort_search`) stems from the prefetcher pulling in unused cache lines that evict useful entries.

Table II summarizes the CPI deltas (`no_prefetch` minus `prefetch`) for every benchmark ending in `.c`. Positive values indicate that prefetching improved performance.

Overall, lightweight sequential prefetching provides sub-

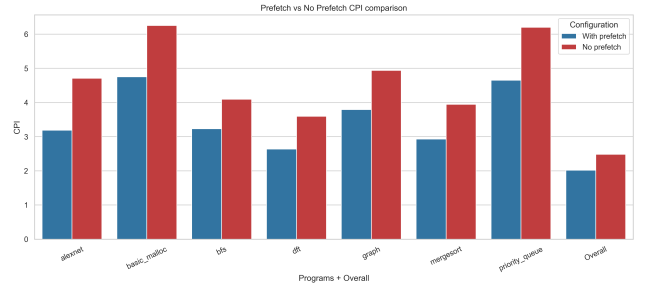


Fig. 12. CPI comparison for configurations with and without prefetching.

Program	Δ CPI
alexnet	1.525
backtrack	0.674
basic_malloc	1.499
bfs	0.862
dft	0.960
fc_forward	1.239
graph	1.148
insertionsort	0.004
matrix_mult_rec	0.128
mergesort	1.016
omegalul	1.459
outer_product	0.056
priority_queue	1.551
quicksort	0.386
saxpy	0.134
sort_search	-0.050

TABLE II
CPI BOOST FOR ALL C PROGRAMS.

stantial gains for memory-bound workloads with predictable access patterns while rarely hurting CPI elsewhere.

E. Impact of DCache Concurrency Level

Intuitively, increasing the number of threads should provide more handlers for LSQ requests, thereby improving throughput. However, as shown in Figure 13, we observed an unexpected performance peak when $N=3$.

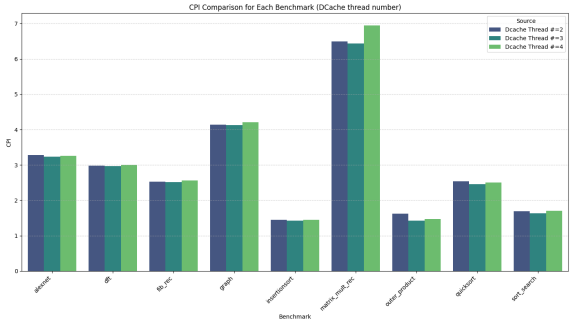


Fig. 13. Performance across different DCache thread counts

One possible explanation is that the “out-of-order” nature of DCache accesses may harm locality, leading to a lower hit rate. From the figure, we can see that as N_THREAD increases, the DCache hit rate gradually decreases, which may account for the performance drop. Increased concurrency can disrupt

the natural access flow of the cache, thereby reducing locality and degrading the hit rate.

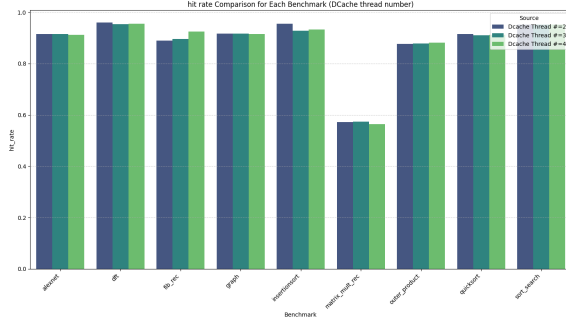


Fig. 14. DCache hit rate across different DCache thread counts

Another observation is that the newly added thread is typically underutilized, so its contribution to overall performance is limited. This is further discussed in Section V.

F. Impact of different LSQ optimizations

From our experiments, LSQ optimization proves to be necessary. The forwarding optimization noticeably reduces CPI, whereas the performance improvement from speculation is less apparent, as shown in Figure 15. One possible explanation is that although speculation helps utilize the waiting time for address conflict detection, the requests that “jump the line” are more likely to incur cache misses, thereby offering limited improvement to the overall hit rate. This also correlates with the results shown in Figure 13. The overall hit rate of LSQ forwarding is summarized in Table III. The relationship between the LSQ forward rate and the DCache hit rate will be further discussed in Section V. It is reasonable to consider LSQ and DCache as a whole since their behavior impacts each other.

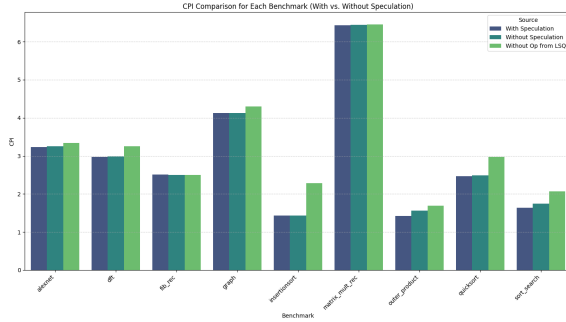


Fig. 15. DCache hit rate comparison when LSQ optimization is added or not. No speculation means only allow forwarding

G. Impact of Branch Unit Count

Figure 16 shows CPI for configurations with NUM_BRANCH=1 and NUM_BRANCH=2. The two curves are nearly identical across all benchmarks. Even branch-heavy workloads such as graph and fib_rec show no meaningful improvement from a second branch unit,

Program	LSQ Forward Rate
alexnet	0.079748
dft	0.295217
fib_rec	0.079307
graph	0.212682
insertionsort	0.625065
matrix_mult_rec	0.118545
outer_product	0.161375
quicksort	0.409757
sort_search	0.428837

TABLE III
LSQ FORWARD RATE FOR EACH BENCHMARK PROGRAM

indicating that branch FU throughput is not a bottleneck in our pipeline.

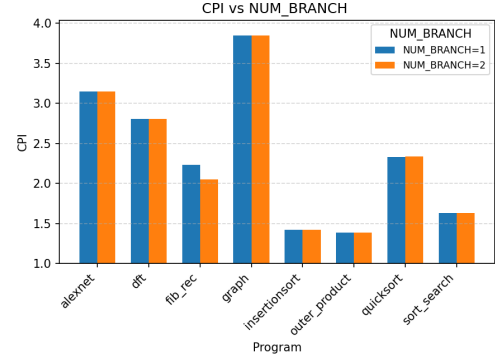


Fig. 16. CPI comparison for branch unit configurations.

The weighted CPI comparison confirms this:

$$\text{WCPI}(1 \text{ BU}) = 2.131, \quad \text{WCPI}(2 \text{ BU}) = 2.129.$$

The 0.002 difference is statistically insignificant, indicating that additional branch functional units do not meaningfully improve IPC for this workload mix.

H. Impact of Multiplier Unit Count

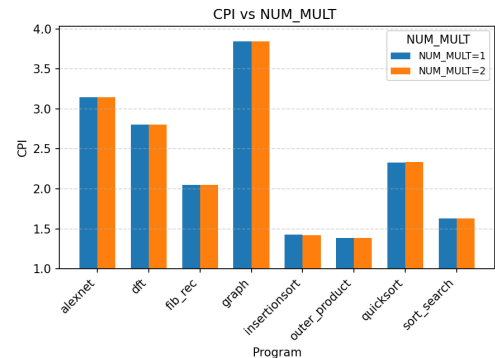


Fig. 17. CPI comparison for multiplier configurations.

Figure 17 compares CPI for NUM_MULT=1 and NUM_MULT=2. The CPI curves almost exactly overlap. Even compute-heavy benchmarks such as outer_product

and alexnet show no measurable advantage from an additional multiplier.

Weighted CPI values likewise show negligible difference:

$$\text{WCPI}(1 \text{ MULT}) = 2.128, \quad \text{WCPI}(2 \text{ MULT}) = 2.129.$$

This demonstrates that the pipelined multiplier sustains throughput effectively with just one unit.

V. DISCUSSION

A. LSQ & DCache Hit Rate Versus CPI

The relationships of the LSQ forwarding hit rate and DCache hit rate is listed in figure 18. They are selected to compare with performance because they both are very important matrix and should give an insight into how to further improve on performance.

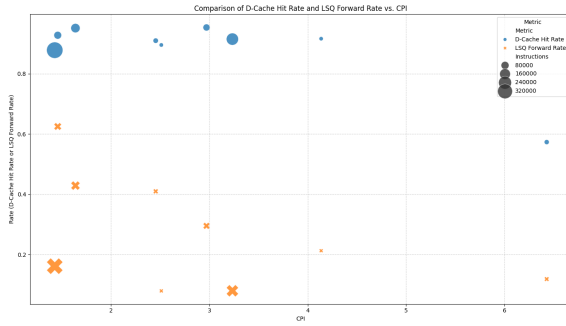


Fig. 18. DCache hit rate and LSQ forwarding hit rate vs. CPI. The bubble size indicate the size of the program. The blue bubbles are DCache hit rates and the Yellow Crossings are LSQ Forward hit rate

From the figure we can see that there is a clear trend that the higher the hit rate of dcache access is the better the performance is. This proves that the data cache access is indeed an important aspect in performance, mostly determinant. However there is also some outliers that have relative low CPI and low hit rate. The reason might be 1) They are not bounded by data accesses. 2) The miss in DCache impact reduced by LSQ forwarding. Interestingly, we can notice a symmetric distribution in the diagram.

B. DCache Threads workloads

Since we have a non-blocking dcache, each of the threads in dcache present as a handler for lsq requests. The loads of each of them turns out not balanced as shown in Figure 19. This is because: 1) The arbiter is implemented with a *psel_gen* module. This module will constantly favor one particular thread line if free. 2) Lots of times, there only exists one outstanding loads from lsq to dcache. The second reason also explains why adding a thread line will not improve performance.

C. Critical paths

VI. CONCLUSION

This project demonstrates the successful design and implementation of a three-wide, P6-style out-of-order RISC-V

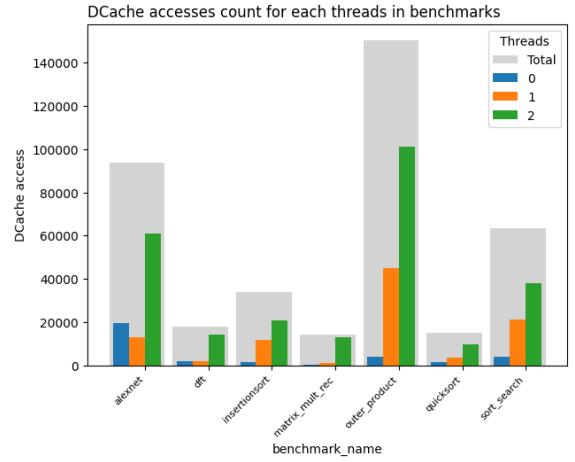


Fig. 19. DCache access through different DCache threads. The sh

processor that balances architectural ambition with practical timing and verification constraints. The final core integrates register renaming, reservation-station scheduling, speculative execution, a non-blocking data cache, aggressive LSQ forwarding, and a hybrid branch predictor, achieving both functional correctness and competitive performance across a diverse benchmark suite.

Post-synthesis results show that the processor meets a minimum clock period of **8.3 ns** while sustaining a weighted average CPI of **2.129**. Sensitivity studies indicate that performance improvements are primarily driven by enlarging the instruction window and improving front-end efficiency through branch prediction and instruction prefetching. In contrast, increasing backend execution resources such as additional branch or multiplier units yields diminishing returns, underscoring the importance of balanced resource provisioning rather than simply widening the pipeline.

Overall, the processor meets the original project goals of delivering a robust, high-performance out-of-order core with advanced memory and control-flow features. Future work could explore deeper and more adaptive branch predictors, early branch recovery, improved cache replacement policies, and tighter LSQ–DCache coordination to further improve memory-level parallelism and hit rates. These extensions would naturally build on the architectural foundation established in this design.